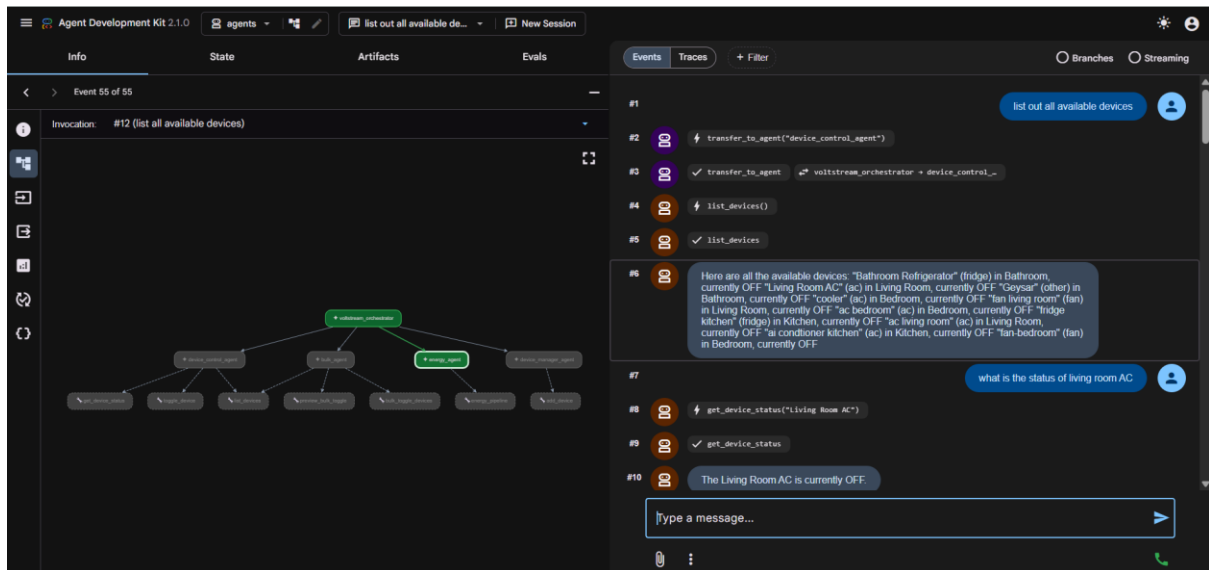


VOLTSTREAM (Week 5)

Intelligent Energy Management Platform

Multi-Agent Architecture using ADK

Source Code: <https://github.com/MYethishwar/VoltStream>



Implemented a multi-agent VoltStream architecture using Google ADK with an orchestrator-based routing system. Developed specialized agents for device control, bulk operations, device management, and energy analytics. Built a sequential energy workflow combining Usage Analyst and Energy Advisor agents with tool chaining, session-state transfer, RAG integration, and HITL confirmation flows. Added decorator-based observability for tool execution, workflow tracing, and debugging.

Project Architecture



Practice Task (Deliverable):

Week- 5 Travel concierge Agent Architecture

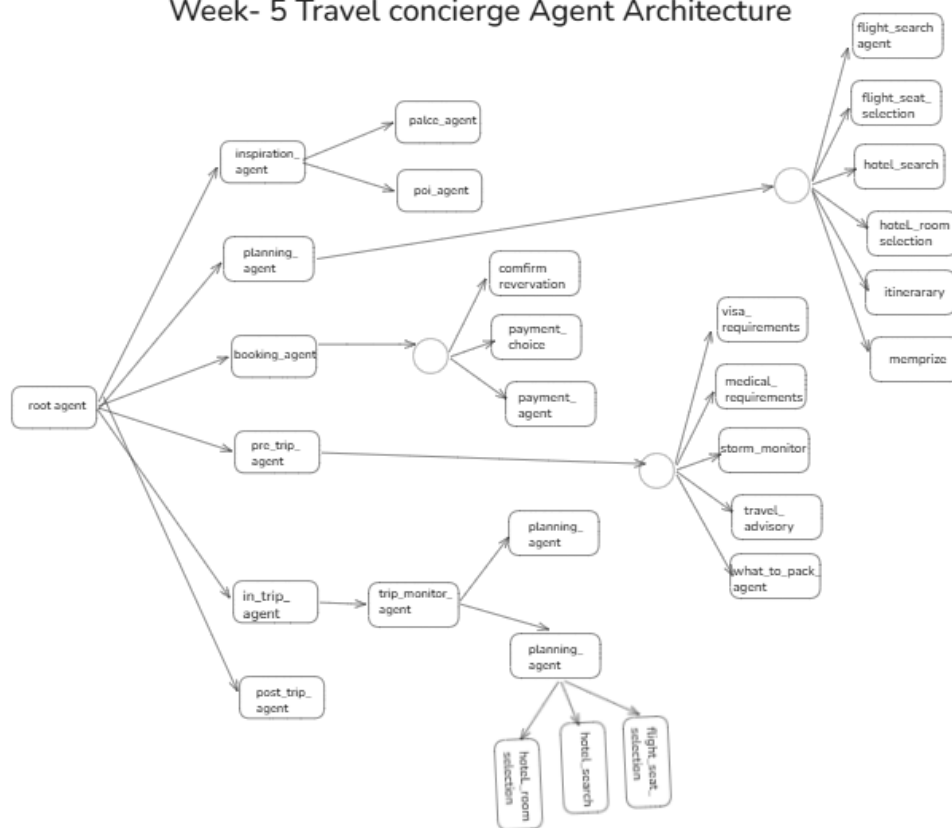


Table of Contents

1. Introduction & Objectives
2. Understanding Project Structure
3. Mult Agent workflow
4. Implementation Details
5. Output & Results

1. Introduction & Objectives

The objective of this task was to design and implement a Multi-Agent Architecture for the VoltStream Smart Energy Management System using Google Agent Development Kit (ADK). The goal was to move beyond a single-agent conversational system and develop a modular architecture where specialized agents handle different categories of user requests. This approach improves scalability, maintainability, task delegation, and response accuracy.

The system was designed around an orchestrator-based architecture where a central agent receives user requests, analyzes intent, and delegates the task to the most appropriate specialized agent. Each agent is responsible for a specific domain and has access only to the tools required for its responsibilities. This separation of concerns ensures that device-related requests, energy analysis tasks, and bulk operations are handled independently while maintaining a unified user experience.

The implemented architecture consists of multiple specialized agents including Device Control Agent, Device Manager Agent, Bulk Operations Agent, and Energy Agent. The Energy Agent further incorporates a Sequential Workflow that performs usage analysis followed by personalized energy recommendations. This design demonstrates how complex business workflows can be represented using interconnected agents and tools.

The primary objectives achieved during this task include:

- Designing a scalable multi-agent architecture using Google ADK.
- Implementing intent-based routing through a central orchestrator.
- Creating specialized agents with dedicated responsibilities.
- Developing a sequential energy analysis workflow.
- Integrating Retrieval-Augmented Generation (RAG) for knowledge-based recommendations.
- Implementing Human-in-the-Loop (HITL) validation for bulk operations.
- Adding observability and execution tracing using Python decorators.

The architecture enables the system to efficiently process user requests while maintaining clear separation between control logic, analytical workflows, database interactions, and recommendation generation.

2. Understanding Project Folder Structure

The project follows a modular architecture where agents, services, tools, and database operations are separated into dedicated modules. This structure improves maintainability and allows individual components to be updated without affecting the entire system.

The VoltStream project follows a modular and scalable architecture designed to support a Multi-Agent AI system for smart home energy management. Rather than placing all functionality inside a single application file, the project is divided into multiple layers including agents, tools, services, database modules, API routes, and frontend components. This structure improves maintainability, debugging, scalability, and future feature development.

The architecture follows the principle of separation of concerns, where each module is responsible for a specific functionality. Agents focus on decision-making and workflow orchestration, tools perform operations and database interactions, services contain business logic, while API routes expose functionalities to the frontend. This organization enables developers to independently modify or extend different parts of the system without affecting the overall application.

Root Project Structure

The project consists of two major layers:

Frontend Layer

The frontend is developed using React.js and serves as the user interaction layer. It allows users to communicate with the AI assistant, manage devices, monitor energy consumption, and access analytics dashboards.

Major frontend responsibilities include:

- User authentication and session management.
- Dashboard visualization.
- Device monitoring and control.
- Energy consumption reporting.
- AI chatbot interaction.
- User settings and profile management.

The frontend communicates with the backend through REST APIs and agent endpoints.

Backend Layer

The backend is developed using FastAPI and serves as the core processing layer of the system.

Major backend responsibilities include:

- Agent execution.
- Workflow orchestration.
- Database operations.
- Device management.
- Energy analytics.

- RAG-based knowledge retrieval.
- User authentication.
- API management.

The backend acts as the bridge between user requests, AI agents, and persistent storage systems.

Agents Directory

The agents directory is the most important component of the multi-agent architecture. It contains all AI agents responsible for handling user requests.

Each agent has a clearly defined responsibility and accesses only the tools required for its domain.

orchestrator.py

The orchestrator acts as the entry point for all user interactions.

Its primary responsibility is intent classification and task delegation.

Whenever a user submits a request, the orchestrator analyzes the request and determines which specialized agent should process it.

Examples:

- Device status requests are delegated to Device Control Agent.
- Device registration requests are delegated to Device Manager Agent.
- Bulk operations are delegated to Bulk Agent.
- Energy-related queries are delegated to Energy Agent.

This design prevents a single agent from handling all responsibilities and improves overall system scalability.

The orchestrator functions similarly to a manager in an organization who assigns tasks to appropriate departments rather than performing every task personally.

device_control.py

This module contains the Device Control Agent.

The Device Control Agent is responsible for:

- Listing devices.
- Checking device status.
- Turning devices ON.
- Turning devices OFF.
- Monitoring individual devices.

The agent interacts directly with device-related tools and database records.

Whenever users request information regarding a specific device, the request is routed to this agent.

Examples:

"List all devices"

"Turn off bedroom AC"

"Check fan status"

The Device Control Agent ensures accurate execution of individual device operations.

device_manager.py

This module contains the Device Manager Agent.

Its responsibility is managing device registration and inventory.

The agent collects required information from users before creating new device records.

The Device Manager Agent validates:

- Device name
- Device room
- Device type

After validation, it calls the appropriate tool to store device information in the database.

Examples:

"Add a new fan"

"Register bedroom AC"

"Create kitchen light"

The Device Manager Agent ensures device data consistency throughout the system.

bulk_agent.py

The Bulk Agent handles operations involving multiple devices simultaneously.

Unlike the Device Control Agent, which manages individual devices, the Bulk Agent performs mass operations.

Examples:

- Turn off all devices.
- Switch off all fans.
- Turn on all lights.
- Power down everything.

To prevent accidental execution, the Bulk Agent implements a Human-in-the-Loop (HITL) workflow.

The workflow follows three stages:

Preview → Confirmation → Execution

Before executing any bulk operation, the agent displays the affected devices and requests user approval.

Only after confirmation does the actual operation occur.

This safety mechanism significantly reduces the risk of unintended actions.

energy_agent.py

The Energy Agent is responsible for handling all energy-related user queries.

Examples include:

- Energy reports
- Usage summaries
- Peak-hour analysis
- Cost estimation
- Energy-saving recommendations

Instead of directly generating responses, the Energy Agent invokes the Energy Pipeline workflow.

This design ensures that recommendations are based on actual usage data rather than assumptions.

energy_pipeline.py

The Energy Pipeline represents a Sequential Agent Workflow.

Unlike normal agents that perform a single operation, the Sequential Agent executes multiple agents in a predefined order.

The pipeline consists of:

1. Usage Analyst Agent
2. Energy Advisor Agent

The Usage Analyst Agent performs data collection and analysis.

The Energy Advisor Agent consumes the analysis and generates recommendations.

This workflow demonstrates one of the most important concepts in Agentic AI systems: multi-step reasoning through coordinated agent execution.

Tools Module

The tools.py module acts as the operational layer of the project.

It contains all executable functions used by agents.

The tools can be categorized into several groups.

Device Tools

These tools manage device interactions.

Examples:

- get_device_status()
- toggle_device()
- list_devices()

These tools are primarily used by the Device Control Agent.

Database Tools

Database tools perform CRUD operations on MongoDB.

Examples:

- `add_device()`
- `update_device_status()`

These tools enable persistent storage and retrieval of device information.

Bulk Operation Tools

These tools handle multiple-device operations.

Examples:

- `preview_bulk_toggle()`
- `bulk_toggle_devices()`

These functions support Human-in-the-Loop validation workflows.

Energy Tools

Energy tools process historical usage information and generate analytics.

Examples:

- `get_usage_history()`
- `get_peak_hours()`
- `get_smart_schedule()`

These tools form the analytical foundation of the energy workflow.

RAG Tools

Retrieval-Augmented Generation tools access external knowledge sources.

Example:

- `get_energy_knowledge()`

These tools retrieve domain-specific energy-saving information and enhance recommendation quality.

Services Directory

The services layer contains business logic and reusable functionality.

Rather than embedding all processing inside agents, services centralize common operations.

Examples include:

- Device management services.
- Energy analytics services.
- RAG services.
- Database interaction services.

This separation improves maintainability and prevents code duplication.

Database Layer

MongoDB serves as the primary storage mechanism.

Collections store:

- Device information.
- Usage history.
- User data.
- Analytics records.

The database enables persistent state management and historical analysis.

Overall Architectural Flow

The complete system workflow can be summarized as follows:

User Request

↓

Frontend

↓

FastAPI Backend

↓

Orchestrator Agent

↓

Specialized Agent

↓

Tools

↓

Services

↓

Database / RAG System

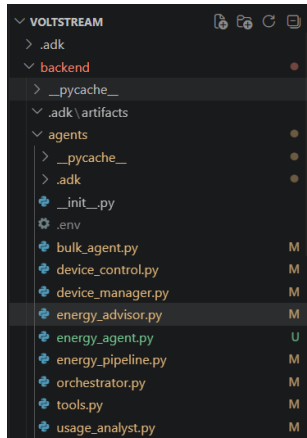
↓

Response Generation

↓

Frontend Display

This modular architecture ensures scalability, maintainability, security, and efficient execution of complex multi-agent workflows while supporting future expansion of the VoltStream platform.



3. Multi-Agent Workflow

The implemented workflow follows an orchestrator-based architecture. The orchestrator acts as the entry point for every user interaction and determines which specialized agent should process the request.

3. Detailed Multi-Agent Workflow Execution

The VoltStream system follows an Orchestrator-Based Multi-Agent Architecture where every user request is first received by a central routing agent called the Orchestrator. The orchestrator analyzes the user's intent and determines which specialized agent should process the request. This approach ensures that each agent focuses only on its assigned domain while improving overall system scalability and maintainability.

The workflow execution begins whenever a user submits a natural language query through the application interface. Instead of directly responding, the orchestrator examines the request and routes it to the appropriate agent. The selected agent then invokes the required tools and generates a response based on the retrieved information.

Workflow 1: Device Control Workflow

The Device Control Workflow is triggered whenever the user requests information or actions related to a single device.

Typical prompts include:

- List all devices
- Show registered devices
- Is the bedroom AC turned on?
- Check refrigerator status
- Turn off living room fan
- Switch on kitchen light

When such requests are detected, the Orchestrator routes them to the Device Control Agent.

Expected workflow:

User Query

↓

Orchestrator Agent

↓

Device Control Agent

↓

Device Tools

↓

Database

↓

Response

The Device Control Agent uses tools such as:

- `list_devices()`
- `get_device_status()`
- `toggle_device()`

For example, when the user enters:

"List all devices"

The workflow becomes:

User

↓

Orchestrator

↓

Device Control Agent

↓

`list_devices()`

↓

MongoDB

↓

Device Information

↓

Response

Similarly, when the user requests:

"Turn off bedroom AC"

The workflow becomes:

User



Orchestrator



Device Control Agent



toggle_device()



Update Device Status



Response

This workflow ensures that individual device operations remain isolated from other system functionalities.

Workflow 2: Device Registration Workflow

The Device Registration Workflow is responsible for managing new device creation and inventory management.

Typical prompts include:

- Add a new fan
- Register bedroom AC
- Add a kitchen light
- Create a new smart device

These requests are delegated by the Orchestrator to the Device Manager Agent.

Expected workflow:

User



Orchestrator



Device Manager Agent



Information Collection



add_device()

↓

Database

↓

Response

The Device Manager Agent validates:

- Device Name
- Device Type
- Device Room

If any required information is missing, the agent requests additional details from the user before executing the database operation.

Example:

User: "Add a fan"

Device Manager Agent:

"What room should the fan be installed in?"

User:

"Bedroom"

Agent:

"What would you like to name the device?"

User:

"Bedroom Fan"

↓

add_device()

↓

Device Created

↓

Response

This workflow ensures data consistency and prevents incomplete device registrations.

Workflow 3: Bulk Operations Workflow

The Bulk Operations Workflow is used whenever the user intends to control multiple devices simultaneously.

Typical prompts include:

- Turn off all devices
- Switch off all fans

- Turn on all lights
- Power down everything

Unlike individual device control, bulk operations incorporate Human-in-the-Loop (HITL) validation to prevent accidental execution.

Expected workflow:

User



Orchestrator



Bulk Agent



preview_bulk_toggle()



Preview Response



User Confirmation



bulk_toggle_devices()



Database Updates



Response

Example:

User:

"Turn off all fans"



Orchestrator



Bulk Agent



preview_bulk_toggle()



Displays affected devices

↓

"Do you want to continue?"

↓

User: "Yes"

↓

bulk_toggle_devices()

↓

Update Device States

↓

Response

The Human-in-the-Loop checkpoint acts as a safety mechanism and ensures that bulk actions occur only after explicit user approval.

Workflow 4: Energy Analysis Workflow

The Energy Analysis Workflow represents the most advanced workflow within the system.

It is triggered whenever users request information related to energy consumption, electricity costs, usage reports, or optimization recommendations.

Typical prompts include:

- Generate energy report
- Show energy usage for last week
- Which device consumes the most power?
- What are my peak usage hours?
- Analyze my electricity consumption
- How can I reduce my electricity bill?

The Orchestrator routes these requests to the Energy Agent.

Expected workflow:

User

↓

Orchestrator

↓

Energy Agent

↓

Energy Pipeline

↓

Usage Analyst Agent

↓

Energy Advisor Agent

↓

Response

Unlike normal agents, the Energy Pipeline uses a Sequential Agent architecture where multiple agents execute in a predefined order.

Stage 1: Usage Analyst Agent

The Usage Analyst Agent performs factual analysis.

Tools used:

- `get_usage_history()`
- `get_peak_hours()`

Responsibilities:

- Calculate device runtime
- Calculate power consumption
- Calculate energy cost
- Identify highest consuming devices
- Detect peak usage periods

The generated output is stored in workflow state and passed to the next agent.

Stage 2: Energy Advisor Agent

The Energy Advisor Agent consumes the analysis generated in Stage 1.

Tools used:

- `get_energy_knowledge()`
- `get_smart_schedule()`

Responsibilities:

- Generate personalized recommendations
- Suggest energy optimization strategies
- Create smart schedules
- Retrieve domain knowledge through RAG
- Produce actionable energy-saving advice

This sequential execution ensures recommendations are generated using actual user data rather than generic assumptions.

Example Workflow:

User:

"Generate a complete energy report with recommendations"

↓

Orchestrator

↓

Energy Agent

↓

Usage Analyst Agent

↓

get_usage_history()

↓

get_peak_hours()

↓

usage_analysis

↓

Energy Advisor Agent

↓

get_energy_knowledge()

↓

get_smart_schedule()

↓

Final Report

↓

Response

Workflow Monitoring and Observability

To improve visibility and debugging, the system implements decorator-based tracing mechanisms.

Each tool invocation can be categorized and monitored using decorators.

Examples include:

- Device Tools
- Energy Tools
- Database Tools
- Bulk Tools

- RAG Tools
- HITL Checkpoints

This observability layer helps developers verify routing decisions, monitor tool execution, validate workflow progression, and troubleshoot issues during development and testing.

The complete workflow architecture demonstrates how multiple specialized agents can collaborate to solve complex tasks through delegation, sequential execution, tool invocation, retrieval augmentation, and human validation mechanisms.

4. Implementation Details

4. Implementation Details

The implementation phase focused on developing a modular Multi-Agent Architecture using Google Agent Development Kit (ADK). The system was designed using specialized agents, custom tools, sequential workflows, Human-in-the-Loop validation mechanisms, and Retrieval-Augmented Generation (RAG) integration. Each component was implemented independently and later integrated into a unified architecture capable of handling multiple categories of user requests.

4.1 Orchestrator Agent Implementation

The Orchestrator Agent serves as the central decision-making component of the system. Every user request first reaches the orchestrator, which analyzes the intent and delegates the task to the most appropriate specialized agent.

Sample Implementation:

```
orchestrator = Agent(
    name="voltstream_orchestrator",
    model="gemini-2.5-flash",
    description="VoltStream root orchestrator.",
    instruction=""
```

Route requests to the appropriate agent.

Device Control → device_control_agent

Bulk Operations → bulk_agent

Energy Queries → energy_agent

Device Management → device_manager_agent

```

"""
    sub_agents=[
        device_control_agent,
        bulk_agent,
        energy_agent,
        device_manager_agent,
    ],
)

```

The orchestrator eliminates the need for a single monolithic agent and allows specialized agents to focus only on their assigned responsibilities.

Benefits

- Improved scalability
- Better prompt management
- Clear separation of responsibilities
- Easier maintenance
- Enhanced workflow visibility

4.2 Device Control Agent Implementation

The Device Control Agent is responsible for handling all single-device operations.

Examples:

- Device status checks
- Turning devices ON/OFF
- Listing devices

The agent is connected to dedicated device management tools.

Sample Implementation:

```

device_control_agent = Agent(
    name="device_control_agent",
    model="gemini-2.5-flash",
    description="Controls individual devices.",
    instruction="""
Handle single device operations.

```

Examples:

- *Turn off AC*
- *Check fan status*
- *List devices*

```
"""
    tools=[
        get_device_status,
        toggle_device,
        list_devices,
    ],
)
```

Associated Tool Examples:

```
def get_device_status(device_name: str):
    ...

def toggle_device(device_name: str, state: bool):
    ...

def list_devices():
    ...
```

The agent communicates with MongoDB through tool functions and returns structured responses to users.

4.3 Device Manager Agent Implementation

The Device Manager Agent is responsible for device registration and inventory management.

Before adding a device, the agent validates:

- Device Name
- Device Type
- Device Room

Sample Implementation:

```
device_manager_agent = Agent(
    name="device_manager_agent",
    model="gemini-2.5-flash",
    tools=[
        add_device
    ]
)
```

)

Tool Implementation:

```
def add_device(  
    device_name: str,  
    room: str,  
    device_type: str  
):  
    ...
```

The tool stores the device information in MongoDB and ensures that duplicate devices are not created.

4.4 Bulk Agent Implementation

The Bulk Agent handles operations affecting multiple devices simultaneously.

Examples include:

- Turn off all devices
- Switch off all fans
- Turn on all lights

A Human-in-the-Loop workflow was implemented to prevent accidental actions.

Sample Implementation:

```
bulk_agent = Agent(  
    name="bulk_agent",  
    model="gemini-2.5-flash",  
    tools=[  
        preview_bulk_toggle,  
        bulk_toggle_devices  
    ]  
)
```

Preview Tool:

```
def preview_bulk_toggle(  
    filter_type,  
    filter_room,  
    state  
):
```

...

Execution Tool:

```
def bulk_toggle_devices(  
    filter_type,  
    filter_room,  
    state  
):  
    ...
```

Workflow:

Preview



User Confirmation



Execution

This implementation ensures operational safety while managing multiple devices.

4.5 Energy Agent Implementation

The Energy Agent is responsible for all energy-related requests.

Examples include:

- Energy reports
- Cost estimation
- Usage analysis
- Optimization recommendations

Unlike traditional agents, the Energy Agent invokes a Sequential Workflow instead of directly generating responses.

Sample Implementation:

```
energy_agent = Agent(  
    name="energy_agent",  
    model="gemini-2.5-flash",  
    tools=[  
        AgentTool(energy_pipeline)  
    ]  
)
```

This architecture enables the system to perform multi-stage reasoning before generating recommendations.

4.6 Sequential Energy Pipeline

The Energy Pipeline is one of the most important components of the project.

A Sequential Agent executes multiple agents in a predefined order.

Pipeline Structure:

Usage Analyst Agent



Energy Advisor Agent

Implementation:

```
energy_pipeline = SequentialAgent(  
    name="energy_pipeline",  
    sub_agents=[  
        usage_analyst_agent,  
        energy_advisor_agent,  
    ],  
)
```

This workflow ensures that energy recommendations are based on real usage analysis rather than assumptions.

4.7 Usage Analyst Agent

The Usage Analyst Agent performs factual analysis of historical energy consumption data.

Tools Used:

- `get_usage_history()`
- `get_peak_hours()`

Sample Implementation:

```
usage_analyst_agent = Agent(  
    name="usage_analyst_agent",  
    model="gemini-2.5-flash",  
    tools=[  
        get_usage_history,  
        get_peak_hours,
```

```
],  
    output_key="usage_analysis",  
)  
  
Usage History Tool:  
  
def get_usage_history(days: int = 7):
```

...

Peak Hour Analysis Tool:

def get_peak_hours():

...

Responsibilities:

- Runtime calculation
- Energy consumption analysis
- Cost estimation
- Peak-hour identification

The generated output is stored in session state and passed to the next workflow stage.

4.8 Energy Advisor Agent

The Energy Advisor Agent consumes the analysis produced by the Usage Analyst Agent.

Tools Used:

- *get_energy_knowledge()*
- *get_smart_schedule()*

Sample Implementation:

```
energy_advisor_agent = Agent(  
    name="energy_advisor_agent",  
    model="gemini-2.5-flash",  
    tools=[  
        get_energy_knowledge,  
        get_smart_schedule,  
    ],  
)
```

The agent accesses the workflow state using:

```
{usage_analysis}
```

Responsibilities:

- Recommendation generation
 - Smart scheduling
 - Cost optimization
 - Personalized energy-saving advice
-

4.9 Retrieval-Augmented Generation (RAG)

The project integrates RAG to improve recommendation quality.

Instead of relying entirely on the LLM's internal knowledge, the system retrieves information from external energy knowledge sources.

Sample Tool:

```
def get_energy_knowledge(  
    query: str  
):  
    return get_rag_response(query)
```

Benefits:

- More accurate recommendations
- Domain-specific knowledge
- Up-to-date guidance
- Reduced hallucinations

RAG significantly improves the reliability of generated energy-saving suggestions.

4.10 Tool Categorization and Observability

To improve debugging and workflow monitoring, decorators were introduced for tool categorization.

Example Decorator:

```
def energy_tool(func):  
    def wrapper(*args, **kwargs):  
        print(  
            f"ENERGY TOOL: {func.__name__}"  
        )  
        return func(*args, **kwargs)  
    return wrapper
```

Usage:

@energy_tool

def get_usage_history():

...

Additional categories include:

@device_tool

@bulk_tool

@db_tool

@rag_tool

@hitl_tool

These decorators provide execution visibility without modifying the core business logic.

Example Output

ENERGY TOOL: get_usage_history

ENERGY TOOL: get_peak_hours

RAG TOOL: get_energy_knowledge

ENERGY TOOL: get_smart_schedule

This observability mechanism greatly simplifies debugging and helps verify workflow execution during development.

Summary

The implementation successfully combined multiple agentic AI concepts including orchestration, delegation, sequential workflows, Human-in-the-Loop validation, Retrieval-Augmented Generation, tool invocation, state sharing, and observability. Together, these components form a scalable and maintainable Multi-Agent AI architecture capable of handling smart home energy management tasks efficiently.

5. Output & Results

The implemented multi-agent architecture successfully demonstrated coordinated task execution across multiple specialized agents. The orchestrator correctly routed user requests to their respective agents based on intent classification.

Device management operations were successfully executed through dedicated device agents. Users were able to retrieve device information, control individual devices, and register new smart devices through natural language interactions.

Bulk operations were executed through a controlled workflow incorporating Human-in-the-Loop validation. The preview-and-confirmation mechanism ensured operational safety and reduced the possibility of accidental actions.

The Energy Pipeline demonstrated successful sequential execution. The Usage Analyst Agent retrieved historical usage information and generated analytical summaries, while the Energy Advisor Agent transformed those insights into actionable recommendations. This confirmed successful state transfer between sequential workflow stages.

The Retrieval-Augmented Generation component successfully enhanced recommendation quality by incorporating external energy-saving knowledge into the advisory process. As a result, recommendations became more contextual and aligned with real-world energy optimization practices.

The observability layer provided clear visibility into workflow execution. Tool-level tracing allowed verification of routing decisions, sequential workflow progression, database interactions, and RAG retrieval operations. This significantly improved debugging efficiency and workflow validation.

Overall, the project successfully achieved the objective of developing a scalable, modular, and extensible multi-agent architecture capable of handling device management, energy analytics, recommendation generation, and controlled bulk operations. The final implementation demonstrates how modern agentic AI systems can be structured using orchestrators, specialized agents, sequential workflows, retrieval systems, and human approval mechanisms to solve complex real-world tasks efficiently.

Agent Development Kit 2.1.0agentslist out all available de...New Session

InfoStateArtifactsEvals

Event 55 of 55

Invocation: #12 (list all available devices)



#1list out all available devices

#2transfer_to_agent("device_control_agent")

#3transfer_to_agent → volkstream_orchestrator → device_control_...

#4list_devices()

#5list_devices

#6Here are all the available devices: "Bathroom Refrigerator" (fridge) in Bathroom, currently OFF "Living Room AC" (ac) in Living Room, currently OFF "Geysar" (other) in Bathroom, currently OFF "cooler" (ac) in Bedroom, currently OFF "fan living room" (fan) in Living Room, currently OFF "ac bedroom" (ac) in Bedroom, currently OFF "fridge kitchen" (fridge) in Kitchen, currently OFF "ac living room" (ac) in Living Room, currently OFF "ai conditioner kitchen" (ac) in Kitchen, currently OFF "fan-bedroom" (fan) in Bedroom, currently OFF

#7what is the status of living room AC

#8get_device_status("Living Room AC")

#9get_device_status

#10The Living Room AC is currently OFF

type a message...

Agent Development Kit 2.1.0agentslist out all available de...New Session

InfoStateArtifactsEvals

Event 58 of 58

Invocation: #12 (list all available devices)



#25turn on kitchen AC

#26toggle_device

#27toggle_device

#28The kitchen AC has been turned ON

#29turn off all devices

#30transfer_to_agent("bulk_agent")

#31transfer_to_agent → device_control_agent → bulk_agent

#32preview_bulk_toggle

#33preview_bulk_toggle

#34The following 11 device(s) will be turned OFF:

- "Bathroom Refrigerator" | fridge | Bathroom | currently OFF
- "Living Room AC" | ac | Living Room | currently OFF
- "Geysar" | other | Bathroom | currently OFF
- "cooler" | ac | Bedroom | currently OFF
- "fan living room" | fan | Living Room | currently OFF
- "ac bedroom" | ac | Bedroom | currently OFF
- "fridge kitchen" | fridge | Kitchen | currently OFF

Type a message...

Agent Development Kit 2.1.0agentslist out all available de...New Session

InfoStateArtifactsEvals

<>Event 58 of 58

Invocation: #12 (list all available devices)

graph TD; subgraph subgraph_artifact [subgraph_artifact]; direction TB; device_manager_agent[device_manager_agent]; bulk_agent[bulk_agent]; energy_agent[energy_agent]; device_control_agent[device_control_agent]; end; device_manager_agent --> bulk_agent; device_manager_agent --> energy_agent; device_manager_agent --> device_control_agent; bulk_agent --> get_device_status[get_device_status]; bulk_agent --> toggle_device[toggle_device]; energy_agent --> get_device_status; energy_agent --> toggle_device; device_control_agent --> get_device_status; device_control_agent --> toggle_device; end

EventsTraces+ FilterBranchesStreaming

#35yes you can proceed

#36⚡ bulk_toggle_devices

#37✓ bulk_toggle_devices

#38Bulk action complete: 11 succeeded, 0 failed. ✓ Bathroom Refrigerator (Bathroom) → OFF ✓ Living Room AC (Living Room) → OFF ✓ Geysar (Bathroom) → OFF ✓ cooler (Bedroom) → OFF ✓ fan living room (Living Room) → OFF ✓ ac bedroom (Bedroom) → OFF ✓ fridge kitchen (Kitchen) → OFF ✓ ac living room (Living Room) → OFF ✓ air conditioner kitchen (Kitchen) → OFF ✓ fan-bedroom (Bedroom) → OFF ✓ kitchen AC (Kitchen) → OFF

#39how much electricity did i use today

#40⚡ transfer_to_agent("energy_agent")

#41✓ transfer_to_agent → bulk_agent + energy_agent

#42⚡ energy_pipeline("how much electricity did..")

#43✓ energy_pipeline → State: usage_analysis

#44Key Finding

Type a message...

Agent Development Kit 2.1.0agentslist out all available de...New Session

InfoStateArtifactsEvals

<>Event 58 of 58

Invocation: #12 (list all available devices)

graph TD; subgraph subgraph_artifact [subgraph_artifact]; direction TB; device_manager_agent[device_manager_agent]; bulk_agent[bulk_agent]; energy_agent[energy_agent]; device_control_agent[device_control_agent]; end; device_manager_agent --> bulk_agent; device_manager_agent --> energy_agent; device_manager_agent --> device_control_agent; bulk_agent --> get_device_status[get_device_status]; bulk_agent --> toggle_device[toggle_device]; energy_agent --> get_device_status; energy_agent --> toggle_device; device_control_agent --> get_device_status; device_control_agent --> toggle_device; end

EventsTraces+ FilterBranchesStreaming

#45Which device consumes the most energy?

#46⚡ energy_pipeline("which device consumes th..")

#47✓ energy_pipeline → State: usage_analysis

#48The cooler is currently your highest energy-consuming device. Here are some recommendations:

- **Reduce Standby Power:** Unplug appliances when not in use to eliminate standby power waste.
- **Optimize AC Usage:** Increase your AC setpoint temperature, as each degree can save about 6% energy.
- **Smart Scheduling:** Run high-wattage appliances like washing machines or EV chargers during off-peak hours.

Your current schedule looks efficient! No major shifts recommended. For a quick action today, unplug your cooler when it's not in use to save on standby power consumption.

#49generate my energy report

#50⚡ energy_pipeline("generate my energy repor..")

#51✓ energy_pipeline → State: usage_analysis

#52Key Finding

Type a message...

Agent Development Kit 2.1.0agentslist out all available de...New Session

InfoStateArtifactsEvals

<>Event 58 of 58

Invocation: #12 (list all available devices)

graph TD; subgraph subgraph_artifact [subgraph_artifact]; direction TB; device_manager_agent[device_manager_agent]; bulk_agent[bulk_agent]; energy_agent[energy_agent]; device_control_agent[device_control_agent]; end; device_manager_agent --> bulk_agent; device_manager_agent --> energy_agent; device_manager_agent --> device_control_agent; bulk_agent --> get_device_status[get_device_status]; bulk_agent --> toggle_device[toggle_device]; energy_agent --> get_device_status; energy_agent --> toggle_device; device_control_agent --> get_device_status; device_control_agent --> toggle_device; end

EventsTraces+ FilterBranchesStreaming

#12⚡ transfer_to_agent("device_manager_agent")

#13✓ transfer_to_agent → device_control_agent + device_manager_age..

#14What would you like to name the AC in the Kitchen?

#15kitchen-AC

#16⚡ add_device

#17✓ add_device

#18kitchen-AC added to Kitchen

#19what is the status of kitchen-AC

#20⚡ transfer_to_agent("device_control_agent")

#21✓ transfer_to_agent → device_manager_agent + device_control_age..

#22⚡ get_device_status("kitchen-AC")

#23✓ get_device_status

Type a message...